rent_prices_modeling

March 4, 2026

Rent price prediction in Berlin

Modeling and predictions

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
```

[2]:
```
df = pd.read_csv('rent_listings_final.csv')
df.shape
```

[2]:
```
(15601, 22)
```

[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15601 entries, 0 to 15600
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   listing_url           15601 non-null  object
 1   cold_rent             15601 non-null  float64
 2   area                  15601 non-null  float64
 3   rooms                 15601 non-null  float64
 4   floor                 13659 non-null  float64
 5   free_from             15601 non-null  object
 6   has_balkon            15601 non-null  bool
 7   has_terrasse          15601 non-null  bool
 8   has_garten            15601 non-null  bool
 9   elevator              15601 non-null  bool
 10  parking               15601 non-null  bool
 11  has_basement          15601 non-null  bool
 12  is_barrier-free       15601 non-null  bool
 13  has_built-in_kitchen  15601 non-null  bool
```

```
 14  has_bathtub           15601 non-null  bool
 15  has_shower            15601 non-null  bool
 16  flooring_type         15601 non-null  object
 17  energy_source         15601 non-null  object
 18  heating_type          15601 non-null  object
 19  property_condition    15601 non-null  object
 20  year_built            7662 non-null   float64
 21  district              15601 non-null  object
dtypes: bool(10), float64(5), object(7)
memory usage: 1.6+ MB
```

[4]:
```
# So we get reproducable results:
np.random.seed(16)
```

Part 1: Feature Engineering

1. Split the dataset into train set (80%) and test set (20%).

[5]:
```
from sklearn.model_selection import train_test_split

X = df.drop(columns=["cold_rent", "listing_url"])
y = df["cold_rent"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

2. Encoding categorical variables.

[6]:
```
X_train = pd.get_dummies(X_train, drop_first=True)
X_test = pd.get_dummies(X_test, drop_first=True)
```

[7]:
```
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
```

[8]:
```
feature_names = X_train.columns.tolist()
```

[9]:
```
len(feature_names)
```

[9]: 93

93 features for 15601 samples is not too many features.

3. Scaling the data.

[10]:
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(X_train)

X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Part 2: Modeling and model evaluation

1. Modeling and training

[11]:
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", LinearRegression())
])

rmse_scores = cross_val_score(pipeline, X_train, y_train,␣
 ↪scoring="neg_mean_squared_error", cv=10)

mae_scores = cross_val_score(pipeline, X_train, y_train,␣
 ↪scoring="neg_mean_absolute_error", cv=10)

r2_scores = cross_val_score(pipeline, X_train, y_train, scoring="r2", cv=10)

print("RMSE:", np.sqrt(-rmse_scores.mean()))
print("MAE:", -mae_scores.mean())
print("R2:", r2_scores.mean())
```

```
RMSE: 431.52294225249767
MAE: 291.25321240316003
R2: 0.6586293404895679
```

Linear regression performed with an R score of 0.658 which is not very good, but we cannot expect for a simple model to capture a all the trends with such a huge dataset.

[11]:
```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("poly", PolynomialFeatures(include_bias=False)),
    ("model", Ridge())
])

param_grid = {
    "poly__degree": [1, 2],
    "model__alpha": [0.01, 0.1, 1.0, 10.0]
}

grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
```

```
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

print("Best parameters:", grid.best_params_)
print("CV RMSE:", np.sqrt(-grid.best_score_))

y_pred = best_model.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2 = r2_score(y_test, y_pred)
test_mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", test_rmse)
print("Test R2:", test_r2)
print("Test MAE:", test_mae)
```

```
Best parameters: {'model__alpha': 10.0, 'poly__degree': 2}
CV RMSE: 373.1340218336258
Test RMSE: 334.63324642399056
Test R2: 0.7923459250139335
Test MAE: 201.5778182347564
```

A polynomial of 2nd degree performs better much better than the linear model, but with an R squared of 0.792 this is definitely nfar from the best model we could use.

[12]:
```
from sklearn.neighbors import KNeighborsRegressor

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", KNeighborsRegressor(algorithm="ball_tree"))
])

param_grid = {
    "model__n_neighbors": [3, 5],
    "model__weights": ["distance"],
}

grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
```

```
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

print("Best parameters:", grid.best_params_)
print("CV RMSE:", np.sqrt(-grid.best_score_))

y_pred = best_model.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2 = r2_score(y_test, y_pred)
test_mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", test_rmse)
print("Test R2:", test_r2)
print("Test MAE:", test_mae)
```

```
Best parameters: {'model__n_neighbors': 3, 'model__weights': 'distance'}
CV RMSE: 334.1973123520061
Test RMSE: 312.8973176500452
Test R2: 0.8184459401555682
Test MAE: 165.6691937740411
```

KNN performs slighlty better, but not good enough to be considered as our final model.

[16]:
```
from sklearn.svm import SVR
from sklearn.model_selection import RandomizedSearchCV

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", SVR())
])

param_grid_svr = [
    {"model__kernel": ["linear"], "model__C": [0.1, 1, 10]},
    {"model__kernel": ["rbf"], "model__C": [1, 10, 100], "model__gamma":␣
 ↪["scale", "auto"]},
    {"model__kernel": ["poly"], "model__C": [1, 10], "model__gamma": ["scale"],␣
 ↪"model__degree": [2, 3]},
]

grid = RandomizedSearchCV(
    pipeline,
    param_grid_svr,
    n_iter=10,  # we only try 10 random combos instead of all
    cv=4,
```

```
    scoring="neg_mean_squared_error",
    n_jobs=-1,
    random_state=42
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

print("Best parameters:", grid.best_params_)
print("CV RMSE:", np.sqrt(-grid.best_score_))

y_pred = best_model.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2 = r2_score(y_test, y_pred)
test_mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", test_rmse)
print("Test R2:", test_r2)
print("Test MAE:", test_mae)
```

```
Best parameters: {'model__kernel': 'rbf', 'model__gamma': 'auto', 'model__C':
100}
CV RMSE: 411.75446404469386
Test RMSE: 381.8233677760727
Test R2: 0.7296494282005865
Test MAE: 224.70776244182488
```

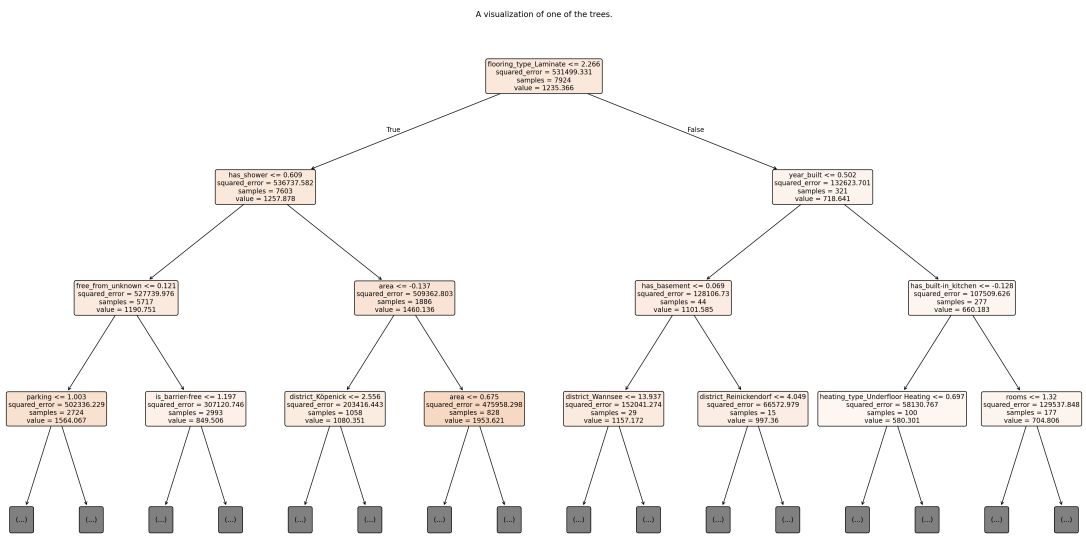
[20]:
```
from sklearn.ensemble import RandomForestRegressor

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", RandomForestRegressor())
])

param_grid_rf = {
    "model__n_estimators": [100, 200, 300, 400],
    "model__max_depth": [None, 10, 20, 30, 40],
    "model__min_samples_split": [2, 5, 10, 20],
    "model__min_samples_leaf": [1, 2, 4, 8],
    "model__max_features": ["sqrt", "log2"],
}

grid = GridSearchCV(
    pipeline,
    param_grid_rf,
```

```
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

print("Best parameters:", grid.best_params_)
print("CV RMSE:", np.sqrt(-grid.best_score_))

y_pred = best_model.predict(X_test)

test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2 = r2_score(y_test, y_pred)
test_mae = mean_absolute_error(y_test, y_pred)

print("Test RMSE:", test_rmse)
print("Test R2:", test_r2)
print("Test MAE:", test_mae)
```

```
Best parameters: {'model__max_depth': 40, 'model__max_features': 'sqrt',
'model__min_samples_leaf': 1, 'model__min_samples_split': 2,
'model__n_estimators': 400}
CV RMSE: 232.59543244821896
Test RMSE: 214.55694132162282
Test R2: 0.9146335103228163
Test MAE: 125.15653759484364
```

We got a good R squared of 0.914 and lower errors. This is the best model so far. We should aim for a higher R squared.

[48]:
```
from sklearn.tree import export_graphviz, plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(30, 15), dpi=200)

plot_tree(
    best_model.named_steps["model"].estimators_[16],
    filled=True,
    max_depth=3,
    feature_names=feature_names,
    rounded=True,
    fontsize=10
)
plt.title("A visualization of one of the trees.")
plt.show()
```

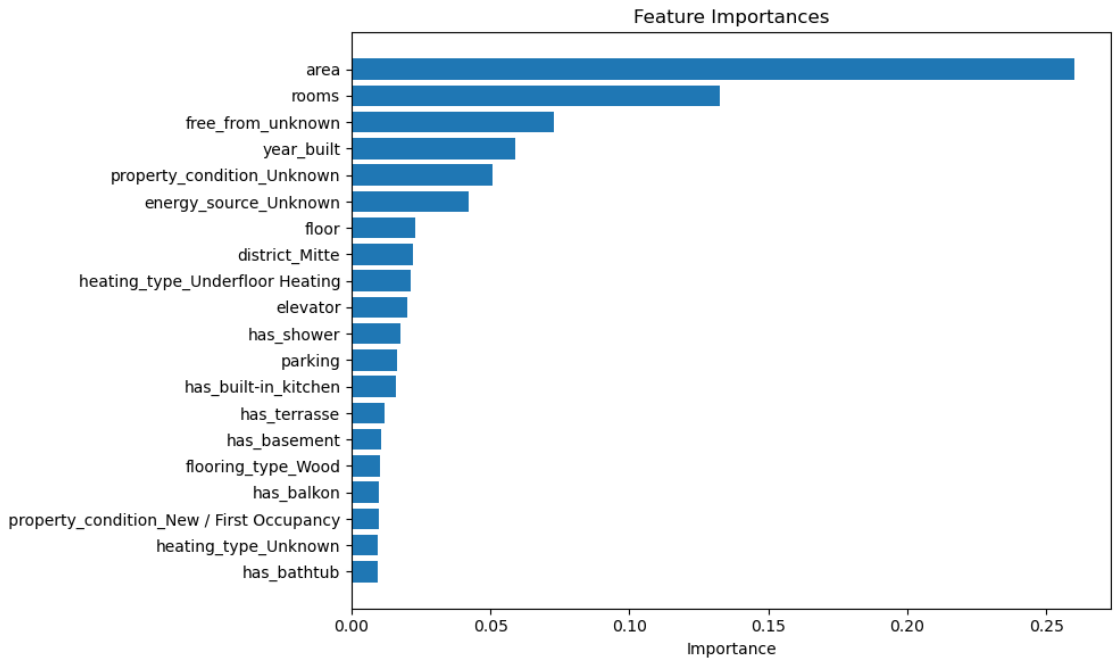


While a couple of things might seem confusing like values of thresholds for each splits due to standardization or the name of the features used due to encoding, the interpretation of the image becomes unclear. The purpuse of this visualization is just to create an idea how the Random Forest is operating by selecting the best features from a random set of features for each split and computing the best thresholds to minimize the loss.

[50]:
```
importances = best_model.named_steps["model"].feature_importances_

fi_df = pd.DataFrame({"feature": feature_names, "importance": importances})
fi_df = fi_df.sort_values("importance", ascending=False)

print(fi_df[:20])

plt.figure(figsize=(10, 6))
plt.barh(fi_df["feature"][:20], fi_df["importance"][:20])
plt.gca().invert_yaxis()
plt.title("Feature Importances")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```

```
                                   feature  importance
0                                     area    0.260227
1                                    rooms    0.132474
15                       free_from_unknown    0.072788
13                              year_built    0.058907
39                 property_condition_Unknown    0.050774
28                      energy_source_Unknown    0.042247
2                                    floor    0.022883
```

```
69                            district_Mitte    0.022057
30          heating_type_Underfloor Heating    0.021198
6                                   elevator    0.019933
12                               has_shower    0.017417
7                                    parking    0.016497
10                     has_built-in_kitchen    0.015918
4                              has_terrasse    0.011835
8                              has_basement    0.010565
20                       flooring_type_Wood    0.010092
3                                 has_balkon    0.009818
34  property_condition_New / First Occupancy    0.009739
32                     heating_type_Unknown    0.009572
11                               has_bathtub    0.009445
```



The graph shows the importance of the features. As we also noticed during Exploratory Data Analysis, the features with the highest imporantace correspond to the features we concluded that had the highest impact, with area having a high importance of 25%, followed by the number of rooms and then the availablity of the apartment. By observing this graph we can gain additional information other than the one we discovered in EDA. For example we have 'free_from_unknown' as our 3rd feature, with a very relatively high importance. What we notice here that many unknown categories show up as the top 15 most imporant features. These could do due to the fact that some of them had high occurances from 35%-65% depending on the feature. During EDA we noticed that some of these categories had a really strong meaning and impact to the price, for example "free_from_unknown" making it to the top 3 here, was associated with a huge negative impact with the price. One specific category, "Miete" being the 8th most important with 2.2% importance shows cleaning and working with difficult features like location really pays of and can

help with modeling for difficult problems.

[14]:
```
from xgboost import XGBRegressor

param_grid_xgb = {
    "model__n_estimators": [400, 600],
    "model__max_depth": [5, 7],
    "model__learning_rate": [0.001, 0.01, 0.1],
    "model__subsample": [0.8, 1.0],
    "model__colsample_bytree": [0.8, 1.0],
    "model__min_child_weight": [1, 3],
    "model__gamma": [0, 0.1]
}

# Model A - With Imputation

pipeline_impute = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", XGBRegressor(random_state=42, objective="reg:squarederror"))
])

grid_impute = GridSearchCV(
    pipeline_impute,
    param_grid_xgb,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid_impute.fit(X_train, y_train)

best_model_impute = grid_impute.best_estimator_

print("Model A (XGBoost with imputation):")
print("Best params:", grid_impute.best_params_)
print("CV RMSE:", np.sqrt(-grid_impute.best_score_))

y_pred_impute = best_model_impute.predict(X_test)

print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_impute)))
print("Test R2:", r2_score(y_test, y_pred_impute))
print("Test MAE:", mean_absolute_error(y_test, y_pred_impute))

# Model B - Without Imputation

pipeline_no_impute = Pipeline([
```

```
    ("model", XGBRegressor(
        random_state=42,
        objective="reg:squarederror"
    ))
])

grid_no_impute = GridSearchCV(
    pipeline_no_impute,
    param_grid_xgb,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid_no_impute.fit(X_train, y_train)

best_model_no_impute = grid_no_impute.best_estimator_

print()
print("Model B (XGBoost without imputation):")
print("Best params:", grid_no_impute.best_params_)
print("CV RMSE:", np.sqrt(-grid_no_impute.best_score_))

y_pred_no_impute = best_model_no_impute.predict(X_test)

print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_no_impute)))
print("Test R2:", r2_score(y_test, y_pred_no_impute))
print("Test MAE:", mean_absolute_error(y_test, y_pred_no_impute))
```

```
Model A (XGBoost with imputation):
Best params: {'model__colsample_bytree': 0.8, 'model__gamma': 0,
'model__learning_rate': 0.1, 'model__max_depth': 7, 'model__min_child_weight':
3, 'model__n_estimators': 600, 'model__subsample': 0.8}
CV RMSE: 212.8904235561409
Test RMSE: 195.44316703340976
Test R2: 0.9291657552651612
Test MAE: 112.50860599086361

Model B (XGBoost without imputation):
Best params: {'model__colsample_bytree': 0.8, 'model__gamma': 0,
'model__learning_rate': 0.1, 'model__max_depth': 7, 'model__min_child_weight':
1, 'model__n_estimators': 600, 'model__subsample': 0.8}
CV RMSE: 213.22053516283677
Test RMSE: 196.07000837411366
Test R2: 0.9287106558292392
Test MAE: 110.4738594543594
```

XGB is one of the few models that can handle missing values, that is the reason why we tried 2

approaches one with imputed missing values and one with missing values as they are. Both models perform nearly the same, with the model with imputed values being sligtly better. Both of them have a really good performance with, an R square of almost 0.93, which considering our data set is a very good and realistic performance. Personally, I thought that XGBoost was going to be the best expected model and most likely this is the model we are going to deploy. If you can notice, the hyperparameters of the best performing model are the higest values, so I would prefer to do another grid search wtih other values, and in case we get a higher performance, we should also check for overfitting.

[13]:
```
param_grid_xgb = {
    "model__n_estimators": [600, 800],
    "model__max_depth": [9, 11],
    "model__learning_rate": [0.1, 0.5],
    "model__subsample": [0.8, 1.0],
    "model__colsample_bytree": [0.8, 1.0],
    "model__min_child_weight": [1, 3],
    "model__gamma": [0, 0.1]
}

# Model A - With Imputation

pipeline_impute = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", XGBRegressor(random_state=42, objective="reg:squarederror"))
])

grid_impute = GridSearchCV(
    pipeline_impute,
    param_grid_xgb,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid_impute.fit(X_train, y_train)

best_model_impute = grid_impute.best_estimator_

print("Model A (XGBoost with imputation):")
print("Best params:", grid_impute.best_params_)
print("CV RMSE:", np.sqrt(-grid_impute.best_score_))

y_pred_impute = best_model_impute.predict(X_test)

print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_impute)))
print("Test R2:", r2_score(y_test, y_pred_impute))
print("Test MAE:", mean_absolute_error(y_test, y_pred_impute))
```

```python
# Model B - Without Imputation

pipeline_no_impute = Pipeline([
    ("model", XGBRegressor(
        random_state=42,
        objective="reg:squarederror"
    ))
])

grid_no_impute = GridSearchCV(
    pipeline_no_impute,
    param_grid_xgb,
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)

grid_no_impute.fit(X_train, y_train)

best_model_no_impute = grid_no_impute.best_estimator_

print()
print("Model B (XGBoost without imputation):")
print("Best params:", grid_no_impute.best_params_)
print("CV RMSE:", np.sqrt(-grid_no_impute.best_score_))

y_pred_no_impute = best_model_no_impute.predict(X_test)

print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_no_impute)))
print("Test R2:", r2_score(y_test, y_pred_no_impute))
print("Test MAE:", mean_absolute_error(y_test, y_pred_no_impute))
```

```
Model A (XGBoost with imputation):
Best params: {'model__colsample_bytree': 0.8, 'model__gamma': 0.1,
'model__learning_rate': 0.1, 'model__max_depth': 9, 'model__min_child_weight':
1, 'model__n_estimators': 600, 'model__subsample': 1.0}
CV RMSE: 211.83437130461814
Test RMSE: 190.97143642129342
Test R2: 0.9323700425641874
Test MAE: 99.74246486283691

Model B (XGBoost without imputation):
Best params: {'model__colsample_bytree': 0.8, 'model__gamma': 0,
'model__learning_rate': 0.1, 'model__max_depth': 9, 'model__min_child_weight':
1, 'model__n_estimators': 600, 'model__subsample': 0.8}
CV RMSE: 211.6443274827569
Test RMSE: 198.73648451087445
```

```
Test R2: 0.9267584559913166
Test MAE: 103.71242791280653
```

We notice a slight improvement by finding a similar combinations of hyperparameters. Looking for another combination of hyperparameters. is pointless since we ended up with almost the same hyperparameters and the improvement is small.

[14]:
```
y_pred_no_impute_train = best_model_no_impute.predict(X_train)

print("Train RMSE:", np.sqrt(mean_squared_error(y_train,␣
 ↪y_pred_no_impute_train)))
print("Train R2:", r2_score(y_train, y_pred_no_impute_train))
print("Train MAE:", mean_absolute_error(y_train, y_pred_no_impute_train))
```

```
Train RMSE: 51.340944444656124
Train R2: 0.9951701756702462
Train MAE: 25.494235073823194
```

[15]:
```
y_pred_impute_train = best_model_impute.predict(X_train)

print("Train RMSE:", np.sqrt(mean_squared_error(y_train, y_pred_impute_train)))
print("Train R2:", r2_score(y_train, y_pred_impute_train))
print("Train MAE:", mean_absolute_error(y_train, y_pred_impute_train))
```

```
Train RMSE: 69.70176935419798
Train R2: 0.9910979253217229
Train MAE: 35.31424842844254
```

If we compare them with the training performance metrics we notice a difference of R squared of 0.06, which is not small, we are undeniably of overfitting and considering XGBoost as an advanced model, overfitting is not uncommon. Considering the difference, the overfitting is not moderate to low and also considering our huge dataset, this level of overfitting is justifiable. Let's also run a RepeatedKFold to check stability, if the model is stable we shouldn't worry.

[16]:
```
from sklearn.model_selection import RepeatedKFold, cross_val_score

best_model = best_model_impute

rkf = RepeatedKFold(
    n_splits=5,
    n_repeats=3,
    random_state=16
)

scores = cross_val_score(
    best_model,
    X_train,
    y_train,
    scoring='neg_root_mean_squared_error',
```

```
    cv=rkf,
    n_jobs=-1
)

rmse_scores = -scores  # make positive

print("Mean RMSE:", np.mean(rmse_scores))
print("Std RMSE:", np.std(rmse_scores))
```

```
Mean RMSE: 210.9995524711056
Std RMSE: 15.83557369099962
```

We have a standard deviation of 15.8, which is not the best we could get (something below 10), but also it is not bad (something above 20). Normally, I would say we are in a gray area, but considering our huge and real dataset, such score is totally realistic and acceptable.

[25]:
```
y_stability_test = np.array(y_test)

# Create dataframe for grouping
df_last_test = pd.DataFrame({
    "y_true": y_stability_test
})

# Quantile boundaries
low_q = np.quantile(y_test, 0.33)
high_q = np.quantile(y_test, 0.66)

def assign_group(value):
    if value <= low_q:
        return "Low"
    elif value <= high_q:
        return "Medium"
    else:
        return "High"

df_last_test["group"] = df_last_test["y_true"].apply(assign_group)

y_pred = best_model.predict(X_test)

df_last_test["y_pred"] = y_pred

group_results = {}

for group_name in ["Low", "Medium", "High"]:
    mask = df_last_test["group"] == group_name

    rmse = np.sqrt(
        mean_squared_error(
```

```
            df_last_test.loc[mask, "y_true"],
            df_last_test.loc[mask, "y_pred"]
        )
    )

    group_results[group_name] = rmse

print("RMSE by target region:")
for k, v in group_results.items():
    print(f"{k}: {v:.3f}")
```

```
RMSE by target region:
Low: 166.490
Medium: 169.581
High: 229.166
```

The model performs well across low and medium target value regions, with a much higher error in the high-value tail due to data skewness and fewer extreme observations. The model can be considered robust and suitable for prediction purposes. I would say it is a very decent model. The only future improvements that could be considered include enhancing the accuracy in high-value samples, but that would be the case if tail prediction is very important for the application and this is not the case.

[48]:
```
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

def build_nn(input_dim):

    model = models.Sequential()

    model.add(layers.Input(shape=(input_dim,)))

    model.add(layers.Dense(256, activation='relu',
             kernel_regularizer=tf.keras.regularizers.l2(0.001)))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.2))

    model.add(layers.Dense(128, activation='relu',
             kernel_regularizer=tf.keras.regularizers.l2(0.001)))
    model.add(layers.BatchNormalization())
    model.add(layers.Dropout(0.2))

    model.add(layers.Dense(64, activation='relu',
             kernel_regularizer=tf.keras.regularizers.l2(0.001)))

    model.add(layers.Dense(32, activation='relu',
             kernel_regularizer=tf.keras.regularizers.l2(0.001)))
```

```
    model.add(layers.Dense(1))  # regression output

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate = 0.0005),
        loss='mse',
        metrics=[tf.keras.metrics.RootMeanSquaredError()]
    )

    return model
```

[49]:
```
nn_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median"))])

X_train_processed = nn_pipeline.fit_transform(X_train)

X_test_processed = nn_pipeline.transform(X_test)
early_stop = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=20,
    restore_best_weights=True
)

nn_model = build_nn(X_train_processed.shape[1])

history = nn_model.fit(
    X_train_processed,
    y_train,
    validation_split=0.2,
    epochs=300,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/300
156/156              3s 5ms/step -
loss: 1978580.1250 - root_mean_squared_error: 1406.6200 - val_loss: 1678356.3750
- val_root_mean_squared_error: 1295.5138
Epoch 2/300
156/156              0s 3ms/step -
loss: 797433.3750 - root_mean_squared_error: 892.9911 - val_loss: 300820.3438 -
val_root_mean_squared_error: 548.4703
Epoch 3/300
156/156              0s 3ms/step -
loss: 261306.3906 - root_mean_squared_error: 511.1806 - val_loss: 220661.0312 -
val_root_mean_squared_error: 469.7448
Epoch 4/300
156/156              0s 3ms/step -
```

```
loss: 208056.1562 - root_mean_squared_error: 456.1308 - val_loss: 183578.4375 -
val_root_mean_squared_error: 428.4596
Epoch 5/300
156/156            1s 3ms/step -
loss: 180187.6719 - root_mean_squared_error: 424.4841 - val_loss: 151940.0000 -
val_root_mean_squared_error: 389.7937
Epoch 6/300
156/156            0s 3ms/step -
loss: 151682.6562 - root_mean_squared_error: 389.4636 - val_loss: 133244.1562 -
val_root_mean_squared_error: 365.0250
Epoch 7/300
156/156            0s 3ms/step -
loss: 134796.0312 - root_mean_squared_error: 367.1446 - val_loss: 120115.0000 -
val_root_mean_squared_error: 346.5748
Epoch 8/300
156/156            1s 3ms/step -
loss: 126572.1797 - root_mean_squared_error: 355.7686 - val_loss: 115124.3359 -
val_root_mean_squared_error: 339.2984
Epoch 9/300
156/156            0s 3ms/step -
loss: 119008.2188 - root_mean_squared_error: 344.9743 - val_loss: 110362.3984 -
val_root_mean_squared_error: 332.2069
Epoch 10/300
156/156            1s 3ms/step -
loss: 115185.0859 - root_mean_squared_error: 339.3878 - val_loss: 107109.5547 -
val_root_mean_squared_error: 327.2745
Epoch 11/300
156/156            1s 4ms/step -
loss: 109790.5391 - root_mean_squared_error: 331.3452 - val_loss: 104728.2188 -
val_root_mean_squared_error: 323.6159
Epoch 12/300
156/156            1s 3ms/step -
loss: 103566.1250 - root_mean_squared_error: 321.8154 - val_loss: 100174.3750 -
val_root_mean_squared_error: 316.5019
Epoch 13/300
156/156            1s 4ms/step -
loss: 102612.9531 - root_mean_squared_error: 320.3311 - val_loss: 96409.9141 -
val_root_mean_squared_error: 310.4979
Epoch 14/300
156/156            1s 3ms/step -
loss: 101361.6016 - root_mean_squared_error: 318.3719 - val_loss: 97273.3750 -
val_root_mean_squared_error: 311.8852
Epoch 15/300
156/156            1s 4ms/step -
loss: 96669.3359 - root_mean_squared_error: 310.9153 - val_loss: 93927.1172 -
val_root_mean_squared_error: 306.4737
Epoch 16/300
156/156            1s 4ms/step -
```

```
loss: 93746.2812 - root_mean_squared_error: 306.1786 - val_loss: 90938.8438 -
val_root_mean_squared_error: 301.5591
Epoch 17/300
156/156            1s 4ms/step -
loss: 92875.9219 - root_mean_squared_error: 304.7539 - val_loss: 90034.9688 -
val_root_mean_squared_error: 300.0566
Epoch 18/300
156/156            1s 4ms/step -
loss: 92176.9219 - root_mean_squared_error: 303.6049 - val_loss: 87751.9219 -
val_root_mean_squared_error: 296.2278
Epoch 19/300
156/156            1s 3ms/step -
loss: 86646.7031 - root_mean_squared_error: 294.3564 - val_loss: 87131.5703 -
val_root_mean_squared_error: 295.1789
Epoch 20/300
156/156            1s 3ms/step -
loss: 88570.5234 - root_mean_squared_error: 297.6063 - val_loss: 85499.7734 -
val_root_mean_squared_error: 292.4017
Epoch 21/300
156/156            0s 3ms/step -
loss: 84549.3047 - root_mean_squared_error: 290.7719 - val_loss: 84174.0859 -
val_root_mean_squared_error: 290.1260
Epoch 22/300
156/156            1s 3ms/step -
loss: 82194.1250 - root_mean_squared_error: 286.6934 - val_loss: 83288.7656 -
val_root_mean_squared_error: 288.5962
Epoch 23/300
156/156            1s 4ms/step -
loss: 81987.9531 - root_mean_squared_error: 286.3336 - val_loss: 81825.5312 -
val_root_mean_squared_error: 286.0498
Epoch 24/300
156/156            1s 4ms/step -
loss: 80333.1250 - root_mean_squared_error: 283.4292 - val_loss: 80615.4688 -
val_root_mean_squared_error: 283.9268
Epoch 25/300
156/156            1s 3ms/step -
loss: 81192.1953 - root_mean_squared_error: 284.9406 - val_loss: 79956.9844 -
val_root_mean_squared_error: 282.7648
Epoch 26/300
156/156            1s 4ms/step -
loss: 78376.7109 - root_mean_squared_error: 279.9566 - val_loss: 77958.4062 -
val_root_mean_squared_error: 279.2085
Epoch 27/300
156/156            1s 4ms/step -
loss: 78951.0078 - root_mean_squared_error: 280.9804 - val_loss: 78709.6484 -
val_root_mean_squared_error: 280.5506
Epoch 28/300
156/156            1s 4ms/step -
```

```
loss: 76497.7734 - root_mean_squared_error: 276.5804 - val_loss: 79329.1562 -
val_root_mean_squared_error: 281.6524
Epoch 29/300
156/156            1s 3ms/step -
loss: 77034.3984 - root_mean_squared_error: 277.5488 - val_loss: 77210.2344 -
val_root_mean_squared_error: 277.8654
Epoch 30/300
156/156            1s 4ms/step -
loss: 74744.4141 - root_mean_squared_error: 273.3923 - val_loss: 76980.7500 -
val_root_mean_squared_error: 277.4521
Epoch 31/300
156/156            1s 3ms/step -
loss: 74553.1641 - root_mean_squared_error: 273.0424 - val_loss: 77651.1484 -
val_root_mean_squared_error: 278.6577
Epoch 32/300
156/156            1s 3ms/step -
loss: 74335.1094 - root_mean_squared_error: 272.6428 - val_loss: 75204.8672 -
val_root_mean_squared_error: 274.2332
Epoch 33/300
156/156            1s 3ms/step -
loss: 72317.6328 - root_mean_squared_error: 268.9174 - val_loss: 75749.0703 -
val_root_mean_squared_error: 275.2236
Epoch 34/300
156/156            1s 3ms/step -
loss: 72564.6875 - root_mean_squared_error: 269.3764 - val_loss: 76719.0547 -
val_root_mean_squared_error: 276.9801
Epoch 35/300
156/156            1s 3ms/step -
loss: 71872.3984 - root_mean_squared_error: 268.0883 - val_loss: 75355.8047 -
val_root_mean_squared_error: 274.5082
Epoch 36/300
156/156            1s 3ms/step -
loss: 69558.3047 - root_mean_squared_error: 263.7370 - val_loss: 73366.3984 -
val_root_mean_squared_error: 270.8604
Epoch 37/300
156/156            0s 3ms/step -
loss: 71407.6797 - root_mean_squared_error: 267.2202 - val_loss: 71851.4609 -
val_root_mean_squared_error: 268.0493
Epoch 38/300
156/156            0s 3ms/step -
loss: 71257.0859 - root_mean_squared_error: 266.9383 - val_loss: 71874.1953 -
val_root_mean_squared_error: 268.0916
Epoch 39/300
156/156            1s 3ms/step -
loss: 67443.8438 - root_mean_squared_error: 259.6974 - val_loss: 73163.9766 -
val_root_mean_squared_error: 270.4864
Epoch 40/300
156/156            0s 3ms/step -
```

```
loss: 67601.4531 - root_mean_squared_error: 260.0007 - val_loss: 71628.1953 -
val_root_mean_squared_error: 267.6324
Epoch 41/300
156/156            1s 5ms/step -
loss: 68762.2656 - root_mean_squared_error: 262.2234 - val_loss: 71425.3516 -
val_root_mean_squared_error: 267.2532
Epoch 42/300
156/156            1s 4ms/step -
loss: 69058.4141 - root_mean_squared_error: 262.7877 - val_loss: 71446.6328 -
val_root_mean_squared_error: 267.2930
Epoch 43/300
156/156            1s 3ms/step -
loss: 67091.3359 - root_mean_squared_error: 259.0179 - val_loss: 69514.4141 -
val_root_mean_squared_error: 263.6538
Epoch 44/300
156/156            1s 3ms/step -
loss: 65282.2109 - root_mean_squared_error: 255.5017 - val_loss: 71289.1016 -
val_root_mean_squared_error: 266.9981
Epoch 45/300
156/156            1s 3ms/step -
loss: 63724.8789 - root_mean_squared_error: 252.4357 - val_loss: 70171.7344 -
val_root_mean_squared_error: 264.8974
Epoch 46/300
156/156            1s 3ms/step -
loss: 64809.7227 - root_mean_squared_error: 254.5754 - val_loss: 69280.0703 -
val_root_mean_squared_error: 263.2090
Epoch 47/300
156/156            1s 3ms/step -
loss: 64331.2812 - root_mean_squared_error: 253.6340 - val_loss: 71374.4219 -
val_root_mean_squared_error: 267.1579
Epoch 48/300
156/156            0s 3ms/step -
loss: 66751.0859 - root_mean_squared_error: 258.3602 - val_loss: 72057.4219 -
val_root_mean_squared_error: 268.4331
Epoch 49/300
156/156            1s 3ms/step -
loss: 64536.3203 - root_mean_squared_error: 254.0378 - val_loss: 69979.0781 -
val_root_mean_squared_error: 264.5335
Epoch 50/300
156/156            1s 3ms/step -
loss: 63405.4805 - root_mean_squared_error: 251.8022 - val_loss: 68663.7031 -
val_root_mean_squared_error: 262.0355
Epoch 51/300
156/156            1s 3ms/step -
loss: 62344.7617 - root_mean_squared_error: 249.6871 - val_loss: 68974.6562 -
val_root_mean_squared_error: 262.6281
Epoch 52/300
156/156            1s 3ms/step -
```

```
loss: 62750.4609 - root_mean_squared_error: 250.4982 - val_loss: 70460.2344 -
val_root_mean_squared_error: 265.4414
Epoch 53/300
156/156            1s 4ms/step -
loss: 62186.7812 - root_mean_squared_error: 249.3706 - val_loss: 67789.0078 -
val_root_mean_squared_error: 260.3611
Epoch 54/300
156/156            0s 3ms/step -
loss: 62600.3086 - root_mean_squared_error: 250.1983 - val_loss: 67536.9688 -
val_root_mean_squared_error: 259.8766
Epoch 55/300
156/156            1s 3ms/step -
loss: 60294.8086 - root_mean_squared_error: 245.5477 - val_loss: 68117.7656 -
val_root_mean_squared_error: 260.9916
Epoch 56/300
156/156            1s 3ms/step -
loss: 61053.2227 - root_mean_squared_error: 247.0872 - val_loss: 67121.8750 -
val_root_mean_squared_error: 259.0767
Epoch 57/300
156/156            1s 3ms/step -
loss: 60740.6172 - root_mean_squared_error: 246.4538 - val_loss: 66962.5312 -
val_root_mean_squared_error: 258.7690
Epoch 58/300
156/156            1s 3ms/step -
loss: 59730.2617 - root_mean_squared_error: 244.3955 - val_loss: 65805.6328 -
val_root_mean_squared_error: 256.5239
Epoch 59/300
156/156            1s 3ms/step -
loss: 61211.5117 - root_mean_squared_error: 247.4073 - val_loss: 68039.8125 -
val_root_mean_squared_error: 260.8422
Epoch 60/300
156/156            1s 3ms/step -
loss: 58810.7070 - root_mean_squared_error: 242.5068 - val_loss: 66717.6797 -
val_root_mean_squared_error: 258.2954
Epoch 61/300
156/156            1s 3ms/step -
loss: 58498.5312 - root_mean_squared_error: 241.8623 - val_loss: 67582.4453 -
val_root_mean_squared_error: 259.9640
Epoch 62/300
156/156            1s 3ms/step -
loss: 59609.9102 - root_mean_squared_error: 244.1491 - val_loss: 67332.2734 -
val_root_mean_squared_error: 259.4825
Epoch 63/300
156/156            1s 3ms/step -
loss: 58050.9023 - root_mean_squared_error: 240.9352 - val_loss: 67503.4922 -
val_root_mean_squared_error: 259.8121
Epoch 64/300
156/156            1s 3ms/step -
```

```
loss: 58310.1602 - root_mean_squared_error: 241.4725 - val_loss: 67000.2188 -
val_root_mean_squared_error: 258.8418
Epoch 65/300
156/156            1s 5ms/step -
loss: 57090.9883 - root_mean_squared_error: 238.9348 - val_loss: 65641.6016 -
val_root_mean_squared_error: 256.2039
Epoch 66/300
156/156            1s 3ms/step -
loss: 58289.4414 - root_mean_squared_error: 241.4297 - val_loss: 67488.1016 -
val_root_mean_squared_error: 259.7825
Epoch 67/300
156/156            0s 3ms/step -
loss: 59377.7891 - root_mean_squared_error: 243.6732 - val_loss: 68240.3047 -
val_root_mean_squared_error: 261.2263
Epoch 68/300
156/156            1s 3ms/step -
loss: 57372.8984 - root_mean_squared_error: 239.5240 - val_loss: 65936.4844 -
val_root_mean_squared_error: 256.7787
Epoch 69/300
156/156            1s 3ms/step -
loss: 58481.5078 - root_mean_squared_error: 241.8270 - val_loss: 66590.6953 -
val_root_mean_squared_error: 258.0494
Epoch 70/300
156/156            1s 4ms/step -
loss: 57613.6875 - root_mean_squared_error: 240.0262 - val_loss: 64996.4570 -
val_root_mean_squared_error: 254.9417
Epoch 71/300
156/156            1s 3ms/step -
loss: 55114.6719 - root_mean_squared_error: 234.7627 - val_loss: 65015.4023 -
val_root_mean_squared_error: 254.9789
Epoch 72/300
156/156            1s 3ms/step -
loss: 57593.2305 - root_mean_squared_error: 239.9835 - val_loss: 63916.4219 -
val_root_mean_squared_error: 252.8147
Epoch 73/300
156/156            1s 3ms/step -
loss: 54988.4102 - root_mean_squared_error: 234.4936 - val_loss: 65438.9375 -
val_root_mean_squared_error: 255.8080
Epoch 74/300
156/156            1s 4ms/step -
loss: 56812.0977 - root_mean_squared_error: 238.3505 - val_loss: 64626.0508 -
val_root_mean_squared_error: 254.2143
Epoch 75/300
156/156            1s 4ms/step -
loss: 55510.9297 - root_mean_squared_error: 235.6051 - val_loss: 64585.1797 -
val_root_mean_squared_error: 254.1338
Epoch 76/300
156/156            1s 3ms/step -
```

```
loss: 55386.0820 - root_mean_squared_error: 235.3400 - val_loss: 64295.9297 -
val_root_mean_squared_error: 253.5640
Epoch 77/300
156/156            0s 3ms/step -
loss: 55596.2773 - root_mean_squared_error: 235.7861 - val_loss: 62933.6797 -
val_root_mean_squared_error: 250.8635
Epoch 78/300
156/156            1s 4ms/step -
loss: 54677.6992 - root_mean_squared_error: 233.8301 - val_loss: 62994.8008 -
val_root_mean_squared_error: 250.9853
Epoch 79/300
156/156            1s 3ms/step -
loss: 54763.7305 - root_mean_squared_error: 234.0140 - val_loss: 63391.1992 -
val_root_mean_squared_error: 251.7738
Epoch 80/300
156/156            1s 4ms/step -
loss: 56328.0117 - root_mean_squared_error: 237.3327 - val_loss: 64454.4375 -
val_root_mean_squared_error: 253.8764
Epoch 81/300
156/156            1s 4ms/step -
loss: 54930.9570 - root_mean_squared_error: 234.3710 - val_loss: 64066.2227 -
val_root_mean_squared_error: 253.1107
Epoch 82/300
156/156            1s 3ms/step -
loss: 54856.9219 - root_mean_squared_error: 234.2130 - val_loss: 63890.5312 -
val_root_mean_squared_error: 252.7634
Epoch 83/300
156/156            1s 4ms/step -
loss: 54241.0000 - root_mean_squared_error: 232.8944 - val_loss: 63663.5312 -
val_root_mean_squared_error: 252.3140
Epoch 84/300
156/156            1s 3ms/step -
loss: 54069.5078 - root_mean_squared_error: 232.5259 - val_loss: 63143.9727 -
val_root_mean_squared_error: 251.2823
Epoch 85/300
156/156            1s 4ms/step -
loss: 53235.7422 - root_mean_squared_error: 230.7260 - val_loss: 65146.1719 -
val_root_mean_squared_error: 255.2351
Epoch 86/300
156/156            1s 3ms/step -
loss: 52451.7305 - root_mean_squared_error: 229.0208 - val_loss: 62509.0430 -
val_root_mean_squared_error: 250.0157
Epoch 87/300
156/156            1s 3ms/step -
loss: 52315.4492 - root_mean_squared_error: 228.7230 - val_loss: 65043.3398 -
val_root_mean_squared_error: 255.0336
Epoch 88/300
156/156            1s 4ms/step -
```

```
loss: 52206.8594 - root_mean_squared_error: 228.4855 - val_loss: 62917.7227 -
val_root_mean_squared_error: 250.8316
Epoch 89/300
156/156            1s 3ms/step -
loss: 50892.5117 - root_mean_squared_error: 225.5910 - val_loss: 62361.5117 -
val_root_mean_squared_error: 249.7204
Epoch 90/300
156/156            1s 3ms/step -
loss: 53038.0273 - root_mean_squared_error: 230.2972 - val_loss: 62586.4805 -
val_root_mean_squared_error: 250.1704
Epoch 91/300
156/156            1s 3ms/step -
loss: 52028.0195 - root_mean_squared_error: 228.0938 - val_loss: 62952.9805 -
val_root_mean_squared_error: 250.9019
Epoch 92/300
156/156            1s 4ms/step -
loss: 52425.7148 - root_mean_squared_error: 228.9640 - val_loss: 63634.7305 -
val_root_mean_squared_error: 252.2568
Epoch 93/300
156/156            1s 3ms/step -
loss: 52288.5508 - root_mean_squared_error: 228.6642 - val_loss: 62419.5781 -
val_root_mean_squared_error: 249.8366
Epoch 94/300
156/156            1s 4ms/step -
loss: 51944.8516 - root_mean_squared_error: 227.9115 - val_loss: 63584.7695 -
val_root_mean_squared_error: 252.1577
Epoch 95/300
156/156            1s 4ms/step -
loss: 51628.5859 - root_mean_squared_error: 227.2165 - val_loss: 61556.0703 -
val_root_mean_squared_error: 248.1024
Epoch 96/300
156/156            1s 3ms/step -
loss: 52623.2891 - root_mean_squared_error: 229.3950 - val_loss: 63937.9727 -
val_root_mean_squared_error: 252.8571
Epoch 97/300
156/156            0s 3ms/step -
loss: 51849.3750 - root_mean_squared_error: 227.7019 - val_loss: 63644.7500 -
val_root_mean_squared_error: 252.2767
Epoch 98/300
156/156            0s 3ms/step -
loss: 51570.7227 - root_mean_squared_error: 227.0892 - val_loss: 62878.1406 -
val_root_mean_squared_error: 250.7527
Epoch 99/300
156/156            0s 3ms/step -
loss: 51994.8125 - root_mean_squared_error: 228.0210 - val_loss: 61127.8281 -
val_root_mean_squared_error: 247.2379
Epoch 100/300
156/156            0s 3ms/step -
```

```
loss: 50599.8750 - root_mean_squared_error: 224.9413 - val_loss: 63611.0703 -
val_root_mean_squared_error: 252.2099
Epoch 101/300
156/156            0s 3ms/step -
loss: 51321.7500 - root_mean_squared_error: 226.5403 - val_loss: 61487.9609 -
val_root_mean_squared_error: 247.9651
Epoch 102/300
156/156            0s 3ms/step -
loss: 53000.2227 - root_mean_squared_error: 230.2151 - val_loss: 61769.3320 -
val_root_mean_squared_error: 248.5319
Epoch 103/300
156/156            1s 4ms/step -
loss: 50460.4531 - root_mean_squared_error: 224.6312 - val_loss: 60226.6094 -
val_root_mean_squared_error: 245.4085
Epoch 104/300
156/156            1s 4ms/step -
loss: 50101.0625 - root_mean_squared_error: 223.8298 - val_loss: 62259.9609 -
val_root_mean_squared_error: 249.5170
Epoch 105/300
156/156            1s 3ms/step -
loss: 51737.0156 - root_mean_squared_error: 227.4549 - val_loss: 60495.7383 -
val_root_mean_squared_error: 245.9562
Epoch 106/300
156/156            1s 3ms/step -
loss: 50160.9102 - root_mean_squared_error: 223.9635 - val_loss: 61370.0273 -
val_root_mean_squared_error: 247.7272
Epoch 107/300
156/156            1s 4ms/step -
loss: 49995.2695 - root_mean_squared_error: 223.5934 - val_loss: 62522.0781 -
val_root_mean_squared_error: 250.0416
Epoch 108/300
156/156            1s 3ms/step -
loss: 49995.3438 - root_mean_squared_error: 223.5935 - val_loss: 61556.6797 -
val_root_mean_squared_error: 248.1036
Epoch 109/300
156/156            1s 4ms/step -
loss: 49184.4023 - root_mean_squared_error: 221.7727 - val_loss: 62239.1992 -
val_root_mean_squared_error: 249.4753
Epoch 110/300
156/156            1s 5ms/step -
loss: 48928.5117 - root_mean_squared_error: 221.1951 - val_loss: 62944.0977 -
val_root_mean_squared_error: 250.8841
Epoch 111/300
156/156            1s 4ms/step -
loss: 51572.1367 - root_mean_squared_error: 227.0922 - val_loss: 61842.0391 -
val_root_mean_squared_error: 248.6780
Epoch 112/300
156/156            1s 4ms/step -
```

```
loss: 51221.9062 - root_mean_squared_error: 226.3197 - val_loss: 65142.8828 -
val_root_mean_squared_error: 255.2285
Epoch 113/300
156/156            1s 4ms/step -
loss: 50506.2695 - root_mean_squared_error: 224.7331 - val_loss: 61965.6523 -
val_root_mean_squared_error: 248.9265
Epoch 114/300
156/156            1s 4ms/step -
loss: 50280.0078 - root_mean_squared_error: 224.2292 - val_loss: 61689.6797 -
val_root_mean_squared_error: 248.3715
Epoch 115/300
156/156            1s 5ms/step -
loss: 49789.0195 - root_mean_squared_error: 223.1317 - val_loss: 62721.6719 -
val_root_mean_squared_error: 250.4404
Epoch 116/300
156/156            1s 5ms/step -
loss: 49750.6953 - root_mean_squared_error: 223.0457 - val_loss: 60698.3711 -
val_root_mean_squared_error: 246.3678
Epoch 117/300
156/156            1s 5ms/step -
loss: 48924.2773 - root_mean_squared_error: 221.1854 - val_loss: 61310.3086 -
val_root_mean_squared_error: 247.6066
Epoch 118/300
156/156            1s 5ms/step -
loss: 47755.3164 - root_mean_squared_error: 218.5269 - val_loss: 60563.8477 -
val_root_mean_squared_error: 246.0946
Epoch 119/300
156/156            1s 4ms/step -
loss: 50312.8789 - root_mean_squared_error: 224.3025 - val_loss: 59718.5703 -
val_root_mean_squared_error: 244.3712
Epoch 120/300
156/156            1s 4ms/step -
loss: 47481.5938 - root_mean_squared_error: 217.8997 - val_loss: 61900.8594 -
val_root_mean_squared_error: 248.7962
Epoch 121/300
156/156            1s 4ms/step -
loss: 47797.0430 - root_mean_squared_error: 218.6224 - val_loss: 61194.8398 -
val_root_mean_squared_error: 247.3733
Epoch 122/300
156/156            1s 3ms/step -
loss: 48254.5586 - root_mean_squared_error: 219.6662 - val_loss: 60382.8672 -
val_root_mean_squared_error: 245.7266
Epoch 123/300
156/156            1s 4ms/step -
loss: 47263.6680 - root_mean_squared_error: 217.3991 - val_loss: 61841.2617 -
val_root_mean_squared_error: 248.6764
Epoch 124/300
156/156            1s 4ms/step -
```

```
loss: 49019.7227 - root_mean_squared_error: 221.4010 - val_loss: 60584.2578 -
val_root_mean_squared_error: 246.1361
Epoch 125/300
156/156            1s 4ms/step -
loss: 46534.2773 - root_mean_squared_error: 215.7150 - val_loss: 60061.0117 -
val_root_mean_squared_error: 245.0708
Epoch 126/300
156/156            1s 4ms/step -
loss: 48613.8672 - root_mean_squared_error: 220.4826 - val_loss: 62275.9023 -
val_root_mean_squared_error: 249.5488
Epoch 127/300
156/156            1s 4ms/step -
loss: 48786.4180 - root_mean_squared_error: 220.8735 - val_loss: 60922.0117 -
val_root_mean_squared_error: 246.8211
Epoch 128/300
156/156            1s 5ms/step -
loss: 48280.2383 - root_mean_squared_error: 219.7246 - val_loss: 59811.3516 -
val_root_mean_squared_error: 244.5609
Epoch 129/300
156/156            1s 5ms/step -
loss: 49478.4531 - root_mean_squared_error: 222.4345 - val_loss: 62315.6016 -
val_root_mean_squared_error: 249.6283
Epoch 130/300
156/156            1s 5ms/step -
loss: 47499.6367 - root_mean_squared_error: 217.9410 - val_loss: 60067.5430 -
val_root_mean_squared_error: 245.0841
Epoch 131/300
156/156            1s 4ms/step -
loss: 46607.2539 - root_mean_squared_error: 215.8841 - val_loss: 59983.3906 -
val_root_mean_squared_error: 244.9124
Epoch 132/300
156/156            1s 4ms/step -
loss: 47143.7070 - root_mean_squared_error: 217.1230 - val_loss: 61300.6602 -
val_root_mean_squared_error: 247.5870
Epoch 133/300
156/156            1s 4ms/step -
loss: 48412.5273 - root_mean_squared_error: 220.0254 - val_loss: 61654.2891 -
val_root_mean_squared_error: 248.3001
Epoch 134/300
156/156            1s 4ms/step -
loss: 47236.2969 - root_mean_squared_error: 217.3361 - val_loss: 61080.1992 -
val_root_mean_squared_error: 247.1414
Epoch 135/300
156/156            1s 4ms/step -
loss: 47731.4375 - root_mean_squared_error: 218.4722 - val_loss: 61124.0312 -
val_root_mean_squared_error: 247.2301
Epoch 136/300
156/156            1s 4ms/step -
```

```
loss: 47104.2969 - root_mean_squared_error: 217.0321 - val_loss: 59547.7227 -
val_root_mean_squared_error: 244.0213
Epoch 137/300
156/156            1s 4ms/step -
loss: 46912.6758 - root_mean_squared_error: 216.5902 - val_loss: 59296.2891 -
val_root_mean_squared_error: 243.5055
Epoch 138/300
156/156            1s 4ms/step -
loss: 47460.6992 - root_mean_squared_error: 217.8517 - val_loss: 59983.2617 -
val_root_mean_squared_error: 244.9120
Epoch 139/300
156/156            1s 4ms/step -
loss: 46951.8945 - root_mean_squared_error: 216.6808 - val_loss: 60867.2422 -
val_root_mean_squared_error: 246.7101
Epoch 140/300
156/156            1s 4ms/step -
loss: 47461.4023 - root_mean_squared_error: 217.8532 - val_loss: 60808.5000 -
val_root_mean_squared_error: 246.5910
Epoch 141/300
156/156            1s 4ms/step -
loss: 46880.6055 - root_mean_squared_error: 216.5162 - val_loss: 59575.2578 -
val_root_mean_squared_error: 244.0777
Epoch 142/300
156/156            1s 4ms/step -
loss: 47226.4805 - root_mean_squared_error: 217.3134 - val_loss: 59783.8008 -
val_root_mean_squared_error: 244.5045
Epoch 143/300
156/156            1s 4ms/step -
loss: 48355.2070 - root_mean_squared_error: 219.8951 - val_loss: 61601.4805 -
val_root_mean_squared_error: 248.1937
Epoch 144/300
156/156            1s 5ms/step -
loss: 46402.1094 - root_mean_squared_error: 215.4084 - val_loss: 60120.0508 -
val_root_mean_squared_error: 245.1911
Epoch 145/300
156/156            1s 4ms/step -
loss: 45317.9023 - root_mean_squared_error: 212.8769 - val_loss: 61504.7422 -
val_root_mean_squared_error: 247.9987
Epoch 146/300
156/156            1s 4ms/step -
loss: 46959.0430 - root_mean_squared_error: 216.6972 - val_loss: 61311.2188 -
val_root_mean_squared_error: 247.6083
Epoch 147/300
156/156            1s 4ms/step -
loss: 46103.8008 - root_mean_squared_error: 214.7148 - val_loss: 65303.9023 -
val_root_mean_squared_error: 255.5436
Epoch 148/300
156/156            1s 4ms/step -
```

```
loss: 45044.7070 - root_mean_squared_error: 212.2341 - val_loss: 60946.8086 -
val_root_mean_squared_error: 246.8713
Epoch 149/300
156/156            1s 4ms/step -
loss: 45713.7188 - root_mean_squared_error: 213.8045 - val_loss: 58881.6289 -
val_root_mean_squared_error: 242.6525
Epoch 150/300
156/156            1s 4ms/step -
loss: 46978.7422 - root_mean_squared_error: 216.7426 - val_loss: 59528.6875 -
val_root_mean_squared_error: 243.9822
Epoch 151/300
156/156            1s 4ms/step -
loss: 44817.5078 - root_mean_squared_error: 211.6982 - val_loss: 59133.5586 -
val_root_mean_squared_error: 243.1711
Epoch 152/300
156/156            1s 5ms/step -
loss: 45656.8125 - root_mean_squared_error: 213.6713 - val_loss: 60086.7500 -
val_root_mean_squared_error: 245.1231
Epoch 153/300
156/156            1s 4ms/step -
loss: 46047.2578 - root_mean_squared_error: 214.5830 - val_loss: 61178.0898 -
val_root_mean_squared_error: 247.3392
Epoch 154/300
156/156            1s 4ms/step -
loss: 46453.2344 - root_mean_squared_error: 215.5269 - val_loss: 58513.5703 -
val_root_mean_squared_error: 241.8929
Epoch 155/300
156/156            1s 5ms/step -
loss: 45896.5156 - root_mean_squared_error: 214.2315 - val_loss: 60670.1680 -
val_root_mean_squared_error: 246.3103
Epoch 156/300
156/156            1s 5ms/step -
loss: 44953.1289 - root_mean_squared_error: 212.0183 - val_loss: 59905.3086 -
val_root_mean_squared_error: 244.7528
Epoch 157/300
156/156            1s 5ms/step -
loss: 44511.5625 - root_mean_squared_error: 210.9743 - val_loss: 60537.3398 -
val_root_mean_squared_error: 246.0405
Epoch 158/300
156/156            1s 5ms/step -
loss: 44574.0078 - root_mean_squared_error: 211.1223 - val_loss: 57821.2070 -
val_root_mean_squared_error: 240.4575
Epoch 159/300
156/156            1s 5ms/step -
loss: 45739.3984 - root_mean_squared_error: 213.8645 - val_loss: 59318.0117 -
val_root_mean_squared_error: 243.5500
Epoch 160/300
156/156            1s 5ms/step -
```

```
loss: 45074.0000 - root_mean_squared_error: 212.3031 - val_loss: 58332.0781 -
val_root_mean_squared_error: 241.5174
Epoch 161/300
156/156            1s 5ms/step -
loss: 45830.7109 - root_mean_squared_error: 214.0778 - val_loss: 59514.8320 -
val_root_mean_squared_error: 243.9537
Epoch 162/300
156/156            1s 4ms/step -
loss: 43263.6758 - root_mean_squared_error: 207.9959 - val_loss: 59473.5195 -
val_root_mean_squared_error: 243.8690
Epoch 163/300
156/156            1s 3ms/step -
loss: 43488.6289 - root_mean_squared_error: 208.5359 - val_loss: 58819.5273 -
val_root_mean_squared_error: 242.5245
Epoch 164/300
156/156            1s 4ms/step -
loss: 44300.5352 - root_mean_squared_error: 210.4736 - val_loss: 59196.2109 -
val_root_mean_squared_error: 243.2998
Epoch 165/300
156/156            1s 4ms/step -
loss: 43800.2578 - root_mean_squared_error: 209.2818 - val_loss: 60155.0586 -
val_root_mean_squared_error: 245.2624
Epoch 166/300
156/156            1s 4ms/step -
loss: 44779.0352 - root_mean_squared_error: 211.6072 - val_loss: 60707.6406 -
val_root_mean_squared_error: 246.3863
Epoch 167/300
156/156            1s 4ms/step -
loss: 45608.6172 - root_mean_squared_error: 213.5584 - val_loss: 59075.3281 -
val_root_mean_squared_error: 243.0513
Epoch 168/300
156/156            1s 4ms/step -
loss: 44474.2070 - root_mean_squared_error: 210.8857 - val_loss: 58514.4297 -
val_root_mean_squared_error: 241.8946
Epoch 169/300
156/156            1s 4ms/step -
loss: 43972.3398 - root_mean_squared_error: 209.6924 - val_loss: 58824.3320 -
val_root_mean_squared_error: 242.5343
Epoch 170/300
156/156            1s 5ms/step -
loss: 44153.1562 - root_mean_squared_error: 210.1231 - val_loss: 58924.5195 -
val_root_mean_squared_error: 242.7407
Epoch 171/300
156/156            1s 5ms/step -
loss: 43474.0117 - root_mean_squared_error: 208.5008 - val_loss: 57970.1328 -
val_root_mean_squared_error: 240.7669
Epoch 172/300
156/156            1s 4ms/step -
```

```
loss: 44214.9844 - root_mean_squared_error: 210.2702 - val_loss: 58187.7812 -
val_root_mean_squared_error: 241.2184
Epoch 173/300
156/156            1s 5ms/step -
loss: 44137.6875 - root_mean_squared_error: 210.0863 - val_loss: 59388.1406 -
val_root_mean_squared_error: 243.6938
Epoch 174/300
156/156            1s 5ms/step -
loss: 43877.0586 - root_mean_squared_error: 209.4650 - val_loss: 58397.6172 -
val_root_mean_squared_error: 241.6530
Epoch 175/300
156/156            1s 5ms/step -
loss: 44630.3320 - root_mean_squared_error: 211.2555 - val_loss: 59560.6094 -
val_root_mean_squared_error: 244.0474
Epoch 176/300
156/156            1s 4ms/step -
loss: 44096.4727 - root_mean_squared_error: 209.9881 - val_loss: 58984.8477 -
val_root_mean_squared_error: 242.8650
Epoch 177/300
156/156            1s 5ms/step -
loss: 44253.4609 - root_mean_squared_error: 210.3616 - val_loss: 58666.7227 -
val_root_mean_squared_error: 242.2091
Epoch 178/300
156/156            1s 4ms/step -
loss: 43424.0977 - root_mean_squared_error: 208.3810 - val_loss: 60066.6211 -
val_root_mean_squared_error: 245.0820
```

[51]:
```
y_pred_nn = nn_model.predict(X_test_processed).flatten()

rmse_nn = np.sqrt(mean_squared_error(y_test, y_pred_nn))
r2_nn = r2_score(y_test, y_pred_nn)

print("NN RMSE:", rmse_nn)
print("NN R2:", r2_nn)
```

```
98/98            0s 945us/step
NN RMSE: 222.38843364601053
NN R2: 0.9082878918325804
```

The Neural Network that we modeled performes better than most of the models, but still it doesn't achieve the results of XGBoost. During training we notice that it converges around 180 epochs, so we cannot improve it further, unless we model a better NN.

[53]:
```
y_pred_nn = nn_model.predict(X_train_processed).flatten()

rmse_nn = np.sqrt(mean_squared_error(y_train, y_pred_nn))
r2_nn = r2_score(y_train, y_pred_nn)
```

```
print("NN RMSE:", rmse_nn)
print("NN R2:", r2_nn)
```

```
390/390              0s 1ms/step
NN RMSE: 177.84509363965668
NN R2: 0.9420454677769694
```

Similar to XGBoost it has a moderate to low level of overfitting, so it doesn't have any advantages over XGBoost.

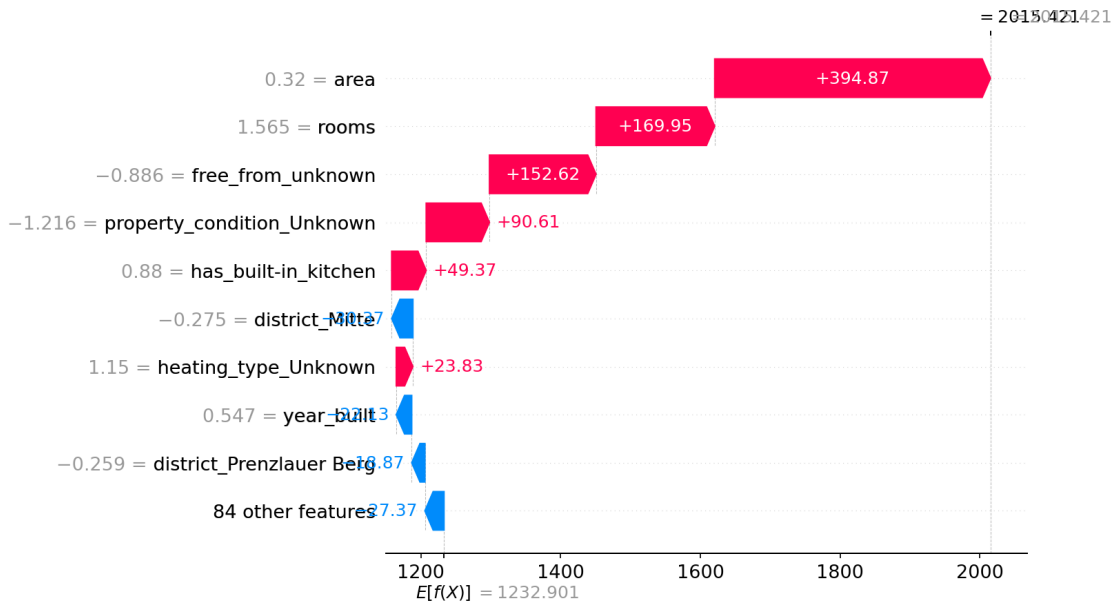
Part 3: Modeling interpretation and model deployment

[23]:
```
best_model
```

[23]:
```
Pipeline(steps=[('imputer', SimpleImputer(strategy='median')),
                ('model',
                 XGBRegressor(base_score=None, booster=None, callbacks=None,
                              colsample_bylevel=None, colsample_bynode=None,
                              colsample_bytree=0.8, device=None,
                              early_stopping_rounds=None,
                              enable_categorical=False, eval_metric=None,
                              feature_types=None, feature_weights=None,
                              gamma=0.1, grow_policy=None, importance_type=None,
                              interaction_constraints=None, learning_rate=0.1,
                              max_bin=None, max_cat_threshold=None,
                              max_cat_to_onehot=None, max_delta_step=None,
                              max_depth=9, max_leaves=None, min_child_weight=1,
                              missing=nan, monotone_constraints=None,
                              multi_strategy=None, n_estimators=600,
                              n_jobs=None, num_parallel_tree=None, …))])
```

[145]:
```
import shap

model = best_model.named_steps["model"]

explainer = shap.Explainer(model)

X_test_processed = best_model.named_steps["imputer"].transform(X_test)

shap_values = explainer(X_test_processed)
```

[147]:
```
explanation = shap.Explanation(
    values=shap_values.values,
    base_values=shap_values.base_values,
    data=X_test_processed,
    feature_names=feature_names
)

shap.plots.waterfall(explanation[0])
```



We start with the expected value of the model's predictions, which is €1232.90. This represents the average predicted rent across all observations, before taking into consideration specific features of a particular apartment. Each feature then contributes to shifting this baseline up or down. The feature with the strongest positive contribution is area, which increases the predicted rent by €394.87. The second most influential feature is number of rooms, which increases the prediction by €169.95. After adding all positive feature contributions, the predicted rent increases to €2015.42. Negative feature contributions then reduce this value to approximately €1150. The feature with the strongest negative contribution is the category "Mitte", which decreases the predicted rent by €30.27, followed by year built, which decreases it by €22.13. This ranking has a high similarity with the feature's importance ranking we displayed above.

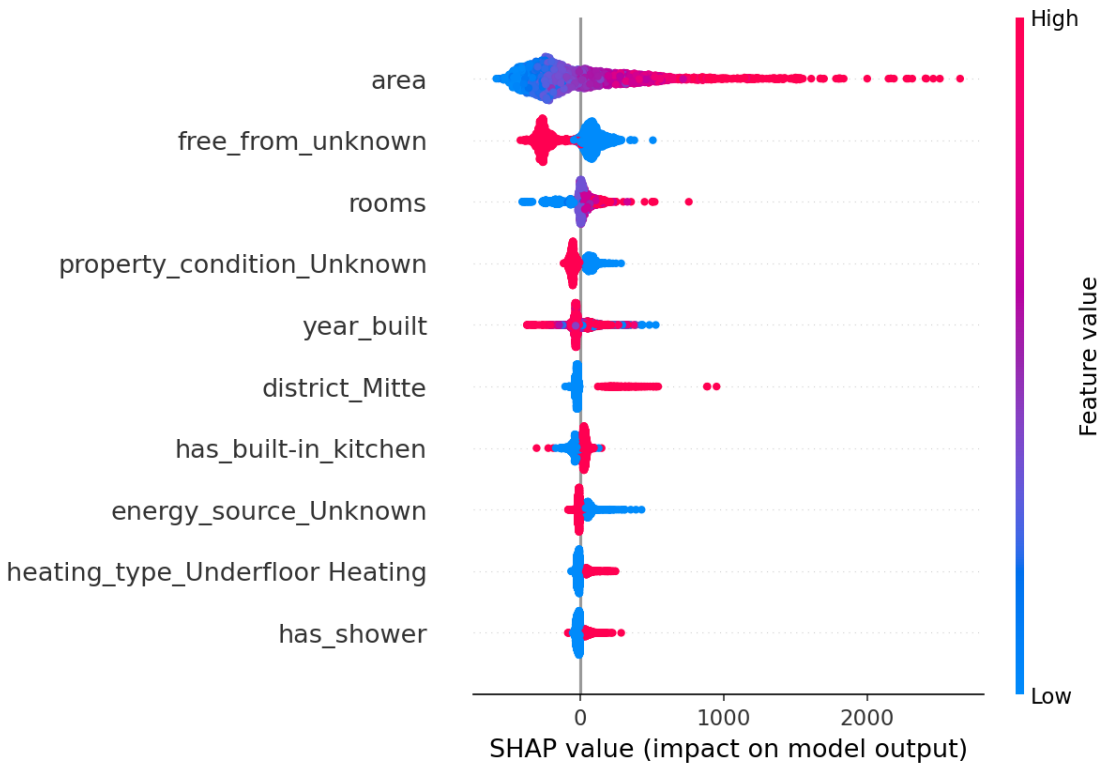
[152]:
```
importances = model.feature_importances_

fi_df = (
    pd.DataFrame({"feature": feature_names, "importance": importances})
    .sort_values("importance", ascending=False)
)

print(fi_df.head(10))
```

```
                              feature  importance
15                   free_from_unknown    0.144185
69                      district_Mitte    0.042682
0                                 area    0.033537
39           property_condition_Unknown    0.030549
79                  district_Schöneberg    0.030335
80                district_Siemensstadt    0.026657
50                   district_Friedenau    0.023540
```

```
33  property_condition_Needs Renovation    0.022501
22                energy_source_Electric    0.020691
28                 energy_source_Unknown    0.020405
```

The importance ranking for our final model does not correspond, which should be expected because they tell us what reduces error, they their role is not for economic interpretation, that is the role of SHAP values, they tell us what moves the predictions.

[167]:
```
import warnings
warnings.filterwarnings("ignore")
shap.summary_plot(explanation, max_display=10)
```



This graph is extremly informative and easy to interpretate. It shows us which feature impact the prices the most and their impact. Feature like area and number of rooms are continous features with a positive contribution, meaning the more we increase them the more the price increases. Features like free_from_unknown, property_condition_unknown, energy_source_unknown are categorical features which have a positive impact when equal to 1 and lower the prices when they are equal to 0 so when they are not present. Year_built is a continous feature with an inverse effect, meaning with the increase of the feature value the prices tend to lower, that would mean the older tend to have lower prices. In this case the effect is not very strong and clear but still visible. Categorical features like "district_Mitte", "has_built-in_kitchen", "heating_type_Underfloor Heating" and "has_shower are features with a positive impact, meaning if present the price is higher.

Save our XGBoost model for deployment

[225]:
```
import joblib

joblib.dump(best_model, "best_model.pkl")
joblib.dump(list(X.columns), "col_names.pkl")
joblib.dump(feature_names, "encoded_col_names.pkl")
joblib.dump(scaler, "scaler.pkl")
```

[225]:
```
['scaler.pkl']
```

[ ]:
```

```

Load the model.

[226]:
```
best_model = joblib.load("best_model.pkl")
new_cols = joblib.load("col_names.pkl")
encoded_cols = joblib.load("encoded_col_names.pkl")
scaler = joblib.load("scaler.pkl")
```

[200]:
```
best_model
```

[200]:
```
Pipeline(steps=[('imputer', SimpleImputer(strategy='median')),
                ('model',
                 XGBRegressor(base_score=None, booster=None, callbacks=None,
                              colsample_bylevel=None, colsample_bynode=None,
                              colsample_bytree=0.8, device=None,
                              early_stopping_rounds=None,
                              enable_categorical=False, eval_metric=None,
                              feature_types=None, feature_weights=None,
                              gamma=0.1, grow_policy=None, importance_type=None,
                              interaction_constraints=None, learning_rate=0.1,
                              max_bin=None, max_cat_threshold=None,
                              max_cat_to_onehot=None, max_delta_step=None,
                              max_depth=9, max_leaves=None, min_child_weight=1,
                              missing=nan, monotone_constraints=None,
                              multi_strategy=None, n_estimators=600,
                              n_jobs=None, num_parallel_tree=None, …))])
```

[201]:
```
new_cols
```

[201]:
```
['area',
 'rooms',
 'floor',
 'free_from',
 'has_balkon',
 'has_terrasse',
 'has_garten',
 'elevator',
 'parking',
```

```
 'has_basement',
 'is_barrier-free',
 'has_built-in_kitchen',
 'has_bathtub',
 'has_shower',
 'flooring_type',
 'energy_source',
 'heating_type',
 'property_condition',
 'year_built',
 'district']
```

[202]:
```
scaler
```

[202]:
```
StandardScaler()
```

[227]:
```
encoded_cols
```

[227]:
```
['area',
 'rooms',
 'floor',
 'has_balkon',
 'has_terrasse',
 'has_garten',
 'elevator',
 'parking',
 'has_basement',
 'is_barrier-free',
 'has_built-in_kitchen',
 'has_bathtub',
 'has_shower',
 'year_built',
 'free_from_not_immediately',
 'free_from_unknown',
 'flooring_type_Laminate',
 'flooring_type_Stone',
 'flooring_type_Tiles',
 'flooring_type_Vinyl',
 'flooring_type_Wood',
 'flooring_type_unknown',
 'energy_source_Electric',
 'energy_source_Gas',
 'energy_source_Heat Pump / Geothermal',
 'energy_source_Hybrid',
 'energy_source_Oil/Coal',
 'energy_source_Other',
 'energy_source_Unknown',
```

```
'heating_type_Individual',
'heating_type_Underfloor Heating',
'heating_type_Unit Heating',
'heating_type_Unknown',
'property_condition_Needs Renovation',
'property_condition_New / First Occupancy',
'property_condition_Old Building',
'property_condition_Other',
'property_condition_Partially Renovated',
'property_condition_Renovated',
'property_condition_Unknown',
'property_condition_Well Maintained',
'district_Altglienicke',
'district_Biesdorf',
'district_Blankenburg',
'district_Bohnsdorf',
'district_Britz',
'district_Buch',
'district_Charlottenburg',
'district_Dahlem',
'district_Französisch Buchholz',
'district_Friedenau',
'district_Friedrichshagen',
'district_Friedrichshain',
'district_Gesundbrunnen',
'district_Grunewald',
'district_Grünau',
'district_Hellersdorf',
'district_Hermsdorf',
'district_Hohenschönhausen',
'district_Johannisthal',
'district_Kladow',
'district_Kreuzberg',
'district_Köpenick',
'district_Lichtenberg',
'district_Lichtenrade',
'district_Lichterfelde',
'district_Mahlsdorf',
'district_Marienfelde',
'district_Marzahn',
'district_Mitte',
'district_Neukölln',
'district_Niederschöneweide',
'district_Niederschönhausen',
'district_Nikolassee',
'district_Oberschöneweide',
'district_Pankow',
```

```
 'district_Prenzlauer Berg',
 'district_Rahnsdorf',
 'district_Reinickendorf',
 'district_Schöneberg',
 'district_Siemensstadt',
 'district_Spandau',
 'district_Staaken',
 'district_Steglitz',
 'district_Tegel',
 'district_Tempelhof',
 'district_Tiergarten',
 'district_Wannsee',
 'district_Wedding',
 'district_Weißensee',
 'district_Wilmersdorf',
 'district_Wittenau',
 'district_Zehlendorf']
```

Test a prediction using unseen data, simulating how a user would interact with the model in production.

We evaluated a new rental listing posted on 03/03/2026 with the following characteristics:

Cold rent: 1,990 euro

Apartment size: 83.3 m^2

Number of rooms: 4

Floor: 1st floor

Availability: immediately

District: Friedrichsfelde, Berlin

Amenities: built-in kitchen, bathtub, and shower

Construction year: 2025

Additional features: balcony, terrace, garden, parking, and basement were absent, while elevator and barrier-free access were available.

[230]:
```
raw_listing = {
    "area": 83.3,
    "rooms": 4,
    "floor": 1,
    "free_from": "immediately",
    "has_balkon": True,
    "has_terrasse": False,
    "has_garten": False,
    "elevator": True,
    "parking": False,
```

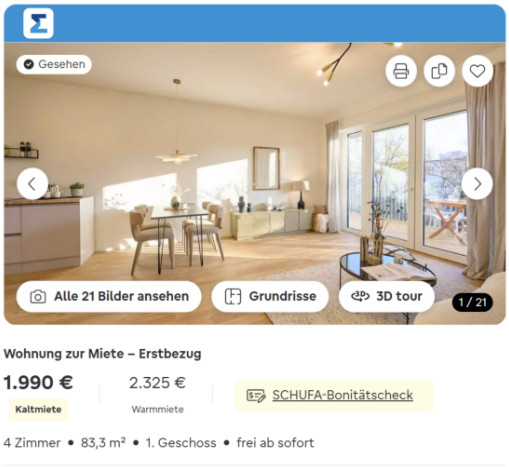
```
    "has_basement": False,
    "is_barrier-free": True,
    "has_built-in_kitchen": True,
    "has_bathtub": True,
    "has_shower": True,
    "flooring_type": "Unknown",
    "energy_source": "District Heating",
    "heating_type": "Underfloor Heating",
    "property_condition": "New / First Occupancy",
    "year_built": 2025,
    "district": "Friedrichsfelde"
}

new_data = pd.DataFrame([raw_listing])

new_data_encoded = pd.get_dummies(new_data, drop_first=True)

# Align columns to training features
new_data_encoded = new_data_encoded.reindex(columns=encoded_cols, fill_value=0)

# Scale
new_data_scaled = scaler.transform(new_data_encoded)

# Predict
prediction = model.predict(new_data_scaled)

print("Prediction:", prediction[0])
```

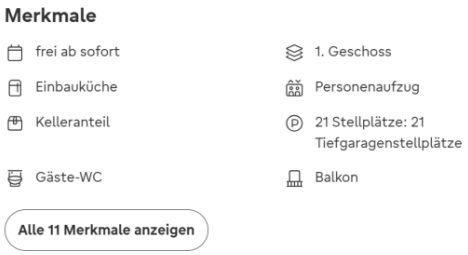
```
Prediction: 1922.7185
```

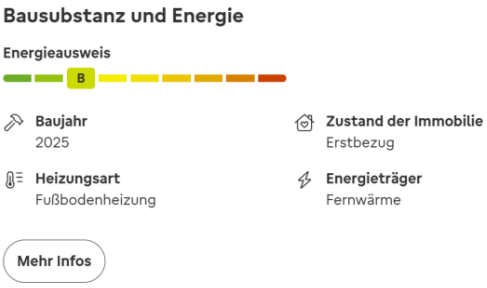
We got an excellent prediction, very close to the original value (1990 euro).

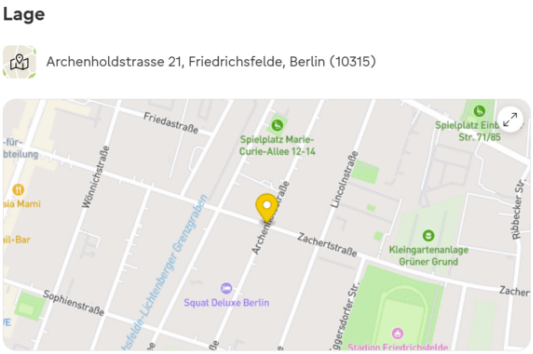
[235]:
```
from PIL import Image

images = ["1.png", "2.png", "3.png", "4.png"]

fig, axes = plt.subplots(2, 2, figsize=(8, 8))

for ax, img_path in zip(axes.flatten(), images):
    img = Image.open(img_path)
    ax.imshow(img)
    ax.axis("off")

fig.suptitle("The original post", fontsize=14)

plt.tight_layout()
plt.show()
```

The original post










```
[ ]:
```